۱- اتصال کیپد و LCD به میکروکنترلر ATMEGA۱۶

برنامه ای بنویسید که شماره متناظر با کلید فشرده شده در کی پد را در LCD نمایش دهد.

شرح گزارش کار :

I. در کدویژن یک پروژه جدید ایجاد میکنیم. در پنجره کد ویزارد چیپ ATMEGA16 را انتخاب میکنیم. برای استفاده از LCD وارد تب Alphanumeric LCD می شویم و آن را Enable میکنیم. چون از LCD 16*2 استفاده میکنیم گزینه Characters/Line را برابر 16 میکنیم. ویزارد به صورت خودکار پورت A را به LCD اختصاص می دهد. برای اتصال کیپد، PORTD0…3 را به عنوان ورودی سطرها با مقاومت بالاکش و PORTD4…7 را به عنوان خروجی (ستون ها) تعیین میکنیم. بقیه تنظیمات را تغییر نمی دهیم. مطمئن میشویم تیک گزینه Program > Generate Code For Disabled Peripherals برداشته شده است. سپس گزینه Program > Generate, Save and Exit را میزنیم. در سه پنجره باز شده برای ذخیره کردن فایل های پروژه همه ی فایل ها را در یک فولدر و در کنار هم ذخیره میکنیم.

II. تابع ()ReadKeypad را با خروجی از نوع char تعریف میکنیم تا کیپد را اسکن کند. نحوه اسکن به این صورت است که ابتدا همه ی سطر ها را برابر 1 میکند (PORTD=0xFF). سپس با شروع از سطر اول، مقدار 0 را در آن مینویسد و بعد بررسی می کند که آیا هیچکدام از ستون های کیپد 0 شده است یا نه. اگر مقداری 0 شده بود، کاراکتر متناظر با آن ستون را بر میگرداند و در غیر اینصورت سطرهای بعدی را به شیوه ای مشابه اسکن می کند. اگر هیچ کلیدی فشرده نشده بود تابع مقدار '' را بر میگرداند. اگر کلید ON/C فشرده شود تابع مقدار 'c' را بر میگرداند. در بالای تابع main، متغیر key از نوع char را برای ذخیره کلید فشرده شده تعریف میکنیم. در داخل حلقه while ابتدا به با دستور lcd_gotoxy(0,0) به خط اول و ستون اول نمایشگر می رویم. سپس با ()ReadKeypad، کیپد را اسکن میکنیم و مقدار آن را در متغیر key ذخیره میکنیم. از آنجایی که ممکن است دکمه ON/C فشرده شده باشد یا اینکه کلیدی فشرده نشده باشد بنابراین دستور شرطی مینویسیم. اگر key=='c' بود با دستور lcd_clear() صفحه پاک می شود ولی در غیر اینصورت اگر key!='' آنگاه با lcd_putchar(key) آن کلید را چاپ می کند. از منوی Project > Build All برنامه را کامپایل میکنیم.

نحوه کار برنامه به این صورت است که کیپد را اسکن میکند و در صورتی که کلیدی فشرده شده باشد آن را روی صفحه چاپ میکند. اگر کلید ON/C فشرده شده باشد، صفحه نمایش را پاک میکند.

III. در پروتئوس یک پروژه جدید با تنظیمات پیش فرض ویزارد ایجاد میکنیم. LCD 16*2 (LM016L)، KEYPAD 4*4 و میکروکنترلر ATMEGA16 را با جستجو در Device ها پیدا میکنیم و به پروژه اضافه میکنیم. VSS را به زمین و VDD را به +5v متصل میکنیم. پایه VEE برای کنترل کنتراست کاراکتر ها استفاده می شود که میتوان با وصل کردن آن به پتانسیومتر مقدار آن را کنترل کرد. طبق سیم کشی ویزارد باید پایه های RS، RW و E را به PORTA0…2 وصل کنیم تا خود کدویژن عملیات کنترلی LCD را مدیریت کند. پایه های دیتا D0…3 را استفاده نمیکنیم و فقط پایه های D4…7 را به PORTA4…7 وصل میکنیم تا انتقال اطلاعات با کمک توابع کتابخانه ای کدویژن صورت پذیرد. سطرهای کیپد را به ترتیب به PORTD0…3 و ستون های آن را به ترتیب به PORTD4…7 وصل میکنیم. روی میکرو دابل کلیک میکنیم. در پنجره باز شده بخش Program File کد هگز برنامه تولید شده در کدویژن را روی میکرو پروگرم میکنیم. با اجرای شبیه ساز مشاهده میکنیم که هرگاه کلیدی فشرده شود کاراکتر مربوط به همان کلید در سر اول و ستون اول LCD چاپ می شود. با فشردن کلیدهای دیگر، کاراکتر مربوط به آن جایگزین کاراکتر قبلی می شود و با فشردن کلید ON/C صفحه نمایش پاک می شود.

۲- نمایش اطلاعات بر روی نمایشگر DOTMATRIX

برنامه ای بنویسید که یک حرف لاتین را روی دات ماتریس نمایش دهد.

شرح گزارش کار :

I. در کدویژن یک پروژه جدید ایجاد میکنیم. در پنجره کد ویزارد چیپ ATMEGA16 را انتخاب میکنیم. برای کنترل دات ماتریس ۸*۸ نیاز به دو پورت خروجی داریم بنابراین PORTA و PORTC را روی حالت خروجی میگذاریم. بقیه تنظیمات را تغییر نمی دهیم. مطمئن میشویم تیک گزینه Program > Generate Code For Disabled Peripherals برداشته شده است. سپس گزینه Program > Generate, Save and Exit را میزنیم. در سه پنجره باز شده برای ذخیره کردن فایل های پروژه همه ی فایل ها را در یک فولدر و در کنار هم ذخیره میکنیم.

II. می دانیم دات ماتریس ۸*۸ از ۸ سطر و ۸ ستون تشکیل شده. آندهای هر سطر همه به هم متصل و کاتدهای هر ستون نیز همه به هم متصلند. به عنوان مثال برای روشن کردن دیود سطر اول و ستون اول باید همه ی سطر ها به جز سطر اول را با مقدار 0 غیرفعال و سطر اول را با 1 فعال کنیم سپس همه ی ستون ها را با 1 غیرفعال و ستون اول را با 0 فعال کنیم. برای نمایش اطلاعات در دات ماتریس یک حلقه ی for از col=0 تا col<8 مینویسیم تا یک به یک ستون های ماتریس را روشن کند. برای اینکار مقدار ~(1<<col) را روی پورت C مینویسیم این دستور اعداد 0b11111110 تا 0b01111111 را روی پورت C می نویسد. مقادیر سطرها را در آرایه row_data ذخیره میکنیم. به این صورت که مثلا برای ستون اول که همه led ها خاموش است مقدار 0x00 و برای ستون دوم مقدار 0xFC(0b11111100) را ذخیره میکنیم. با توجه به شکل حرف 'A' بقیه عناصر آرایه را نیز به همین شکل برای دیگر ستون ها پر میکنیم. برای نمایش این مقادیر روی سطرهای هر ستون مقدار row_data[col] را به پورت A می فرستیم. در پایان برای اینکه چشم متوجه تغییرات حرکت در ستون ها نشود و حرف A را یکجا ببیند، delay_ms(1) را همراه با کتابخانه اش اضافه می کنیم. از منوی Project > Build All برنامه را کامپایل میکنیم.

نحوه کار برنامه به این صورت است که یک به یک ستون های دات ماتریس را از چپ به راست روشن و خاموش میکند و برای هر ستون، سطرهای متناظر که در آرایه row_data مشخص شده اند را روشن میکند. با توجه به مقادیر موجود در row_data حرف 'A' در دات ماتریس مشاهده می شود.

III. در پروتئوس یک پروژه جدید با تنظیمات پیش فرض ویزارد ایجاد میکنیم. MATRIX-8*8 و میکروکنترلر ATMEGA16 را با جستجو در Device ها پیدا میکنیم و به پروژه اضافه میکنیم. برای تشخیص پایه مربوط به هر سطر و ستون به صورت تصادفی پایه های یک طرف را به زمین و پایه های طرف دیگر را به 5V+ وصل میکنیم. اگر چیزی روشن شد یعنی پایه های کاتدها و آندها را تشخیص داده ایم، در غیر اینصورت عکس این حالت سیم کشی می کنیم. برای فهمیدن پایه هر ستون، همه ی پایه های سمت آند را به 5V+ وصل میکنیم و پایه های طرف دیگر را یک به یک به زمین متصل میکنیم. به صورت مشابه برای فهمیدن پایه هر سطر، همه ی پایه های سمت کاتد را به زمین و پایه های طرف دیگر را یک به یک به 5V+ متصل میکنیم. بعد از تشخیص پایه های ماتریس، سطرها را به ترتیب به PORTA0…7 و ستون ها را نیز به ترتیب به PORTC0…7 وصل میکنیم.. روی میکرو دابل کلیک میکنیم. در پنجره باز شده بخش Program File کد هگز برنامه تولید شده در کدویژن را روی میکرو پروگرم میکنیم. با اجرای شبیه ساز مشاهده میکنیم که حرف 'A' روی دات ماتریس نمایش داده می شود.

پیوست ها:

آزمایش ۱)

G:\Personal\Uni\Micro Lab\Micro\Az4-1\Az4-1.c

Notes | Az4-1.c

```
/*****************************************************
This program was created by the
CodeWizardAVR V3.12 Advanced
Automatic Program Generator
© Copyright 1998-2014 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project : Session4-Experiment1
Version :
Date    : 10/27/2020
Author  : Ali Sepehrnia (96121038)
Company :
Comments:


Chip type               : ATmega16
Program type            : Application
AVR Core Clock frequency: 1.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 256
*****************************************************/

#include <mega16.h>
#include <delay.h>

// Alphanumeric LCD functions
#include <alcd.h>

char ReadKeypad()
{
    PORTD=0xff;

    //--Row 1--//
    PORTD.0 = 0;
```

G:\Personal\Uni\Micro Lab\Micro\Az4-1\Az4-1.c

Notes | Az4-1.c

```
    PORTD.0 = 0;
    if(PIND.4 == 0) return '7';
    else if(PIND.5 == 0) return '8';
    else if(PIND.6 == 0) return '9';
    else if(PIND.7 == 0) return '/';
    PORTD.0 = 1;

    //--Row 2--//
    PORTD.1 = 0;
    if(PIND.4 == 0) return '4';
    else if(PIND.5 == 0) return '5';
    else if(PIND.6 == 0) return '6';
    else if(PIND.7 == 0) return '*';
    PORTD.1 = 1;

    //--Row 3--//
    PORTD.2 = 0;
    if(PIND.4 == 0) return '1';
    else if(PIND.5 == 0) return '2';
    else if(PIND.6 == 0) return '3';
    else if(PIND.7 == 0) return '-';
    PORTD.2 = 1;

    //--Row 4--//
    PORTD.3 = 0;
    if(PIND.4 == 0) return 'c';
    else if(PIND.5 == 0) return '0';
    else if(PIND.6 == 0) return '=';
    else if(PIND.7 == 0) return '+';
    PORTD.3 = 1;

    return '';
}

// Declare your global variables here
```

G:\Personal\Uni\Micro Lab\Micro\Az4-1\Az4-1.c

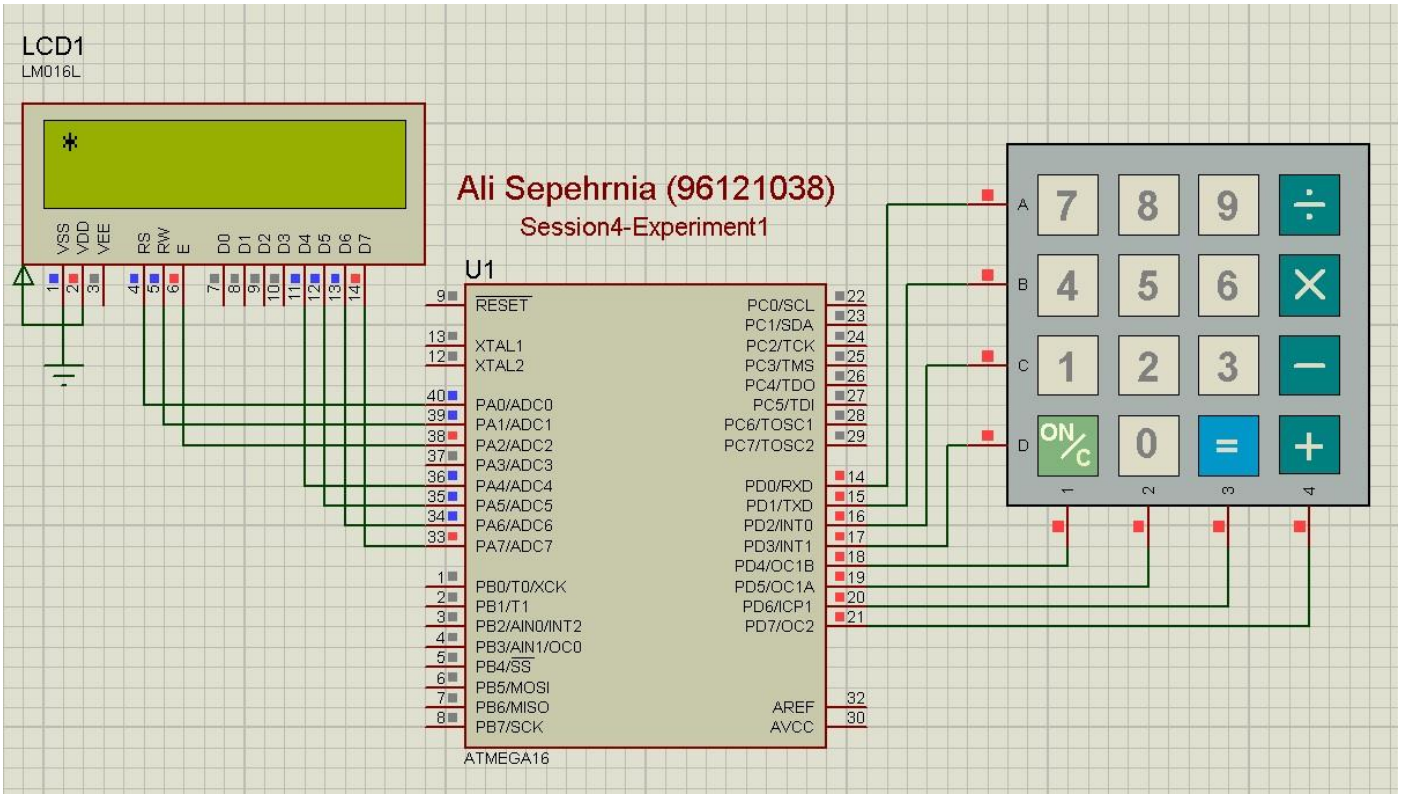
Notes | Az4-1.c

```
71  void main(void)
72  {
73  char key;
74  // Declare your local variables here
75
76  // Input/Output Ports initialization
77  // Port A initialization
78  // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
79  DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
80  // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
81  PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
82
83  // Port B initialization
84  // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
85  DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (0<<DDB4) | (0<<DDB3) | (0<<DDB2) | (0<<DDB1) | (0<<DDB0);
86  // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
87  PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
88
89  // Port C initialization
90  // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
91  DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
92  // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
93  PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);
94
95  // Port D initialization
96  // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=Out Bit2=Out Bit1=Out Bit0=Out
97  DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (1<<DDD3) | (1<<DDD2) | (1<<DDD1) | (1<<DDD0);
98  // State: Bit7=P Bit6=P Bit5=P Bit4=P Bit3=0 Bit2=0 Bit1=0 Bit0=0
99  PORTD=(1<<PORTD7) | (1<<PORTD6) | (1<<PORTD5) | (1<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);
100
101 // Alphanumeric LCD initialization
102 // Connections are specified in the
103 // Project|Configure|C Compiler|Libraries|Alphanumeric LCD menu:
104 // RS - PORTA Bit 0
105 // RD - PORTA Bit 1
```

G:\Personal\Uni\Micro Lab\Micro\Az4-1\Az4-1.c

Notes | Az4-1.c

```
100
101 // Alphanumeric LCD initialization
102 // Connections are specified in the
103 // Project|Configure|C Compiler|Libraries|Alphanumeric LCD menu:
104 // RS - PORTA Bit 0
105 // RD - PORTA Bit 1
106 // EN - PORTA Bit 2
107 // D4 - PORTA Bit 4
108 // D5 - PORTA Bit 5
109 // D6 - PORTA Bit 6
110 // D7 - PORTA Bit 7
111 // Characters/line: 16
112 lcd_init(16);
113
114 while (1)
115       {
116       // Place your code here
117       lcd_gotoxy(0,0);
118       key = ReadKeypad();
119       if(key=='c')
120         lcd_clear();
121       else if(key!='')
122         lcd_putchar(key);
123       }
124 }
125
```



آزمایش ۲)

G:\Personal\Uni\Micro Lab\Micro\Az4-2org\Az4-2org.c

Notes | Az4-2org.c

```
/*****************************************************
This program was created by the
CodeWizardAVR V3.12 Advanced
Automatic Program Generator
© Copyright 1998-2014 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project : Session4-Experiment2
Version :
Date    : 10/27/2020
Author  : Ali Sepehrnia (96121038)
Company :
Comments:


Chip type               : ATmega16
Program type            : Application
AVR Core Clock frequency: 1.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 256
*****************************************************/

#include <mega16.h>
#include <delay.h>

// Declare your global variables here

void main(void)
{
// Declare your local variables here
int col,row_data[] = {0x00,0xFC,0x12,0x11,0x11,0x12,0xFC,0x00};

// Input/Output Ports initialization
// Port A initialization
```

G:\Personal\Uni\Micro Lab\Micro\Az4-2org\Az4-2org.c

Notes | Az4-2org.c

```
// Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
DDRA=(1<<DDA7) | (1<<DDA6) | (1<<DDA5) | (1<<DDA4) | (1<<DDA3) | (1<<DDA2) | (1<<DDA1) | (1<<DDA0);
// State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

// Port B initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (0<<DDB4) | (0<<DDB3) | (0<<DDB2) | (0<<DDB1) | (0<<DDB0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);

// Port C initialization
// Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
DDRC=(1<<DDC7) | (1<<DDC6) | (1<<DDC5) | (1<<DDC4) | (1<<DDC3) | (1<<DDC2) | (1<<DDC1) | (1<<DDC0);
// State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

// Port D initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

while (1)
      {
      // Place your code here
        for(col=0;col<8;col++)
          {
            PORTC=~(1<<col);        //Turn on col'th column
            PORTA=row_data[col];    //col'th Rows Data
            delay_ms(1);
          }
      }
}
```

